

Implémentation de la théorie des files d'attente dans un système de feux tricolores

Projet Système de Contrôle des Feux

11 juin 2025

1 Introduction

Problématique du Trafic Urbain

- Les carrefours sont des points critiques de la circulation urbaine
- La gestion inefficace des feux de circulation cause des embouteillages
- Conséquences :
 - Augmentation du temps de trajet
 - Pollution accrue
 - Impact économique négatif
- Besoin d'un système intelligent s'adaptant aux conditions de trafic réelles

2 Théorie des Files d'Attente

Fondements de la Théorie des Files d'Attente

- Branche des mathématiques qui étudie les temps d'attente dans les systèmes à capacité limitée
- Éléments fondamentaux :
 - Processus d'arrivée (souvent modélisé par un processus de Poisson)
 - Processus de service
 - Nombre de serveurs
 - Discipline de service (FIFO, LIFO, etc.)
 - Capacité du système
- Application parfaite pour les systèmes de trafic routier

Modèle Mathématique M/M/1

- Modèle M/M/1 : Le plus simple des modèles de files d'attente
 - M : Processus d'arrivée markovien (distribution de Poisson)
 - M : Temps de service markovien (distribution exponentielle)
 - 1 : Un seul serveur
- Paramètres clés :
 - λ : Taux d'arrivée moyen
 - μ : Taux de service moyen
 - $\rho = \lambda/\mu$: Intensité du trafic (doit être < 1 pour stabilité)

Théorème de Little $L = \lambda W$ où :

- L : Nombre moyen d'éléments dans le système
- λ : Taux d'arrivée
- W : Temps moyen passé dans le système

Application aux Carrefours 0.5

- Chaque voie d'approche est une file d'attente
- Arrivées de véhicules : processus de Poisson

- Service = passage au feu vert
- Temps d'attente moyen :

$$W = \frac{1}{\mu - \lambda}$$

- Longueur moyenne de la file :

$$L = \frac{\lambda}{\mu - \lambda}$$

Traduction dans le code

2.1 Paramètres et entités

Le code utilise des variables globales pour modéliser le taux d'arrivée (λ), le taux de service (μ), et la file :

Listing 1 – Paramètres du modèle

```
1 let arrivalRate = 0.8; //      (lambda) - taux d'arriv e
2 let serviceRate = 1.2; //      (mu) - taux de service
3 let entities = [];
4 let stats = {
5   totalVehicles: 0,
6   totalWaitTime: 0,
7   totalServiceTime: 0,
8   vehiclesServed: 0,
9   waitTimes: [],
10  queueLengths: { north: [], south: [], east: [], west: [] },
11  currentQueues: { north: 0, south: 0, east: 0, west: 0 }
12 };
```

2.2 Création des véhicules (arrivées)

Chaque véhicule est créé avec une direction et des paramètres individuels. Voici la fonction de création :

Listing 2 – Création d'un véhicule

```
1 function createVehicle(specifiedDirection = null) {
2   const type = vehicleTypes[Math.floor(Math.random() * vehicleTypes.length)];
3   const direction = specifiedDirection || ['north', 'south', 'east', 'west'][
4     Math.floor(Math.random() * 4)];
5   const vehicle = document.createElement('div');
6   vehicle.className = 'vehicle ${type.class}';
7   vehicle.textContent = type.symbol;
8   vehicle.id = 'vehicle-${entityId++}';
9   vehicle.dataset.arrivalTime = Date.now();
10  vehicle.dataset.direction = direction;
11  vehicle.dataset.speed = type.speed * (0.8 + Math.random() * 0.4);
12  vehicle.dataset.serviceTime = type.serviceTime;
13  vehicle.dataset.length = type.length;
14  vehicle.dataset.waiting = 'false';
15  vehicle.dataset.stopTime = Date.now().toString();
16  vehicle.dataset.queuePosition = '0';
17  // Positionnement selon la direction ...
18  document.getElementById('intersection').appendChild(vehicle);
19  entities.push(vehicle);
20  stats.totalVehicles++;
21 }
```

2.3 Gestion de la file et du service

La fonction `canMove` décide si un véhicule peut avancer, selon la position, le feu et la présence d'autres véhicules :

Listing 3 – Gestion de la file et du feu

```
1 function canMove(vehicle) {
2   const pos = getVehiclePosition(vehicle);
3   // ...
4   // Si dans la zone d'arrêt, vérifier le feu
5   if (inStoppingZone) {
6     let canPassLight = true;
7     if ((pos.direction === 'north' || pos.direction === 'south')) {
8       canPassLight = trafficLightState.northSouth === 'green';
9     } else {
10      canPassLight = trafficLightState.eastWest === 'green';
11    }
12    if (!canPassLight) {
13      // Véhicule doit attendre
14      return false;
15    }
16  }
17  // Sinon, avancer
18  return true;
19 }
```

2.4 Calculs statistiques (Little)

Pour afficher les métriques théoriques :

Listing 4 – Calculs théoriques

```
1 function calculateTheoreticalMetrics() {
2   const rho = arrivalRate / serviceRate;
3   const avgQueueLength = (rho * rho) / (1 - rho);
4   const avgWaitTime = avgQueueLength / arrivalRate;
5   const avgSystemTime = 1 / (serviceRate - arrivalRate);
6   return { rho, avgQueueLength, avgWaitTime, avgSystemTime };
7 }
```

3 Conclusion

La théorie des files d'attente permet d'optimiser la gestion du trafic en modélisant l'arrivée et le service des véhicules. Notre implémentation relie directement la théorie (calculs analytiques) et la simulation (animation JS), permettant de visualiser l'effet des paramètres sur la fluidité du trafic.